INTERVIEW PREP SERIES

Java

Interview Questions & Easy Answers

---

52 questions · 27 marked ★ most asked

Covers JDK/JRE/JVM, data types, Strings, OOP pillars, abstract classes vs interfaces, exceptions, collections (ArrayList, HashMap internals), multithreading, memory and garbage collection, Java 8 features (lambdas, streams), and basic Spring Boot. Includes questions reported in Capgemini, IBM, TCS and Infosys technical rounds.

How to use this document

- Questions go from basic to advanced inside each section. Freshers should master the ★ ones first.
- Answers are written in simple words, the way you would say them in an interview. Learn the idea, not the exact sentences.
- Questions are based on technical rounds reported by candidates on GeeksforGeeks, LeetCode Discuss, Glassdoor, Naukri and similar interview-experience pages, plus the standard question banks interviewers draw from.
- Read one section a day and say each answer out loud once. Speaking it is what makes it stick.

Basics

Q1. What are the main features of Java? ★ Most asked

Java is a high-level, object-oriented language. Key features: **platform independent** ("write once, run anywhere" through the JVM), object-oriented, simple (no explicit pointers), secure, robust (strong typing, exceptions, garbage collection), multithreaded, and has a huge standard library.

Q2. What is the difference between JDK, JRE and JVM? ★ Most asked

- **JVM** (Java Virtual Machine): runs Java bytecode (`.class` files) and converts it to machine code. It's platform-specific.
- **JRE** (Java Runtime Environment): JVM + core libraries needed to **run** Java programs.
- **JDK** (Java Development Kit): JRE + development tools like the compiler `javac` and debugger, needed to **write and compile** programs.

Simple formula: JDK = JRE + tools; JRE = JVM + libraries.

Q3. Why is Java platform independent? ★ Most asked

The compiler (`javac`) converts source code into **bytecode**, not machine code. Bytecode runs on any system that has a JVM, and each OS has its own JVM that translates bytecode for that machine.

Q4. Explain `public static void main(String[] args)`. ★ Most asked

- `public`: the JVM can call it from outside the class.
- `static`: it runs without creating an object of the class.
- `void`: it returns nothing.
- `main`: the name the JVM looks for as the starting point.
- `String[] args`: command-line arguments.

Q5. What are the primitive data types in Java?

8 types: `byte` (1 byte), `short` (2), `int` (4), `long` (8), `float` (4), `double` (8), `char` (2, Unicode), `boolean`. Everything else (String, arrays, objects) is a reference type.

Q6. What are wrapper classes and autoboxing?

Wrapper classes (`Integer`, `Double`, `Character`, `Boolean`...) turn primitives into objects, which are needed for collections like `ArrayList<Integer>`. **Autoboxing** converts a primitive to its wrapper automatically (`Integer x = 5;`) and **unboxing** does the reverse.

Q7. What is a class in Java? ★ Most asked

A class is a blueprint that defines the state (fields) and behaviour (methods) of objects. An object is an instance of the class created with `new`.

```
class Student {
  String name;
  Student(String name) { this.name = name; }
  void greet() { System.out.println("Hi " + name); }
}
Student s = new Student("Asha");
```

Interview tip: "What is Java class" was a reported IBM question on Glassdoor. Define it and give this small example.

Q8. What does the `static` keyword mean? ★ Most asked

Static members belong to the **class**, not to any object, so there's one shared copy.

- Static variable: shared by all objects (e.g. a counter of objects created).
- Static method: called with the class name, can't use `this` or instance variables directly.
- Static block: runs once when the class is loaded.
- Static nested class: a nested class that doesn't need an outer object.

Q9. What is the difference between `final`, `finally` and `finalize`? ★ Most asked

- `final` is a keyword: a final variable can't be reassigned, a final method can't be overridden, a final class can't be extended.
- `finally` is a block after try/catch that always runs (used for cleanup).
- `finalize()` was a method called by the garbage collector before destroying an object. It's deprecated and should not be used.

Strings

Q10. Why are Strings immutable in Java? ★ Most asked

Once a String is created, its value can't change; methods like `concat` return a new String. Reasons: **security** (file paths, URLs and DB credentials can't be changed), the **String pool** can safely share one copy, **thread safety**, and safe caching of the hash code for HashMap keys.

Q11. What is the difference between String, StringBuilder and StringBuffer? ★ Most asked

- `String`: immutable; each change creates a new object.
- `StringBuilder`: mutable and fast; not thread-safe. Best for building strings in loops.
- `StringBuffer`: mutable and thread-safe (synchronized), so it's slower.

Q12. What is the String pool? What's the difference between == and `equals()`? ★ Most asked

The String pool is a special area in the heap where string literals are stored once and reused. `==` compares **references** (same object?), while `equals()` compares **content**.

```
String a = "hi";
String b = "hi";
String c = new String("hi");
a == b        // true (same pooled object)
a == c        // false (c is a new object)
a.equals(c)   // true (same content)
```

Q13. Why should you override `hashCode()` when you override `equals()`?

HashMap and HashSet first use `hashCode()` to find a bucket, then `equals()` to compare. If two objects are equal but give different hash codes, they land in different buckets and collections behave wrongly (like duplicates in a HashSet). The rule: equal objects must have equal hash codes.

OOP

Q14. What are the four pillars of OOP in Java? ★ Most asked

- **Encapsulation**: private fields with public getters/setters, hiding internal data.
- **Abstraction**: hiding implementation details using abstract classes and interfaces.
- **Inheritance**: a class reuses another with `extends`.

- **Polymorphism**: one method behaving differently, through overloading (compile time) and overriding (runtime).

Q15. What is the difference between an abstract class and an interface? ★ Most asked

- An abstract class can have abstract and concrete methods, instance variables, and constructors. An interface traditionally had only abstract methods and `public static final` constants; since Java 8 it can have `default` and `static` methods, and since Java 9 `private` methods.
- A class can **extend only one** abstract class but **implement many** interfaces.
- Use an abstract class for closely related classes sharing code ("is-a"); use an interface for a capability many unrelated classes can have ("can-do", like `Comparable`, `Runnable`).

Interview tip: This was listed in a Capgemini full-stack Java interview experience on GeeksforGeeks.

Q16. Why doesn't Java support multiple inheritance of classes? ★ Most asked

To avoid the **diamond problem**: if a class inherited the same method from two parents, it would be unclear which one to use. Java allows multiple inheritance through interfaces instead; if two default methods clash, the class must override and choose.

Q17. What is the difference between method overloading and overriding? ★ Most asked

- **Overloading**: same method name, different parameters, in the same class. Resolved at compile time.
- **Overriding**: a subclass gives its own version of a parent method with the same signature. Resolved at runtime. Use `@Override`.

Static, final and private methods can't be overridden.

Q18. What is a constructor? Can it be private?

A constructor initialises a new object; it has the class name and no return type. If you don't write one, Java gives a default no-arg constructor. Constructors can be overloaded and can call each other with `this(...)` or the parent with `super(...)`. A private constructor stops other classes from creating objects, which is used in the **Singleton** pattern.

Q19. What is the difference between `this` and `super`?

`this` refers to the current object (its fields, methods or another constructor). `super` refers to the parent class (its fields, methods or constructor). `this()` or `super()` must be the first line of a constructor.

Q20. What are access modifiers in Java?

- `private`: only within the class.
- default (no keyword): within the same package.
- `protected`: same package + subclasses in other packages.
- `public`: everywhere.

Q21. What is the difference between composition, aggregation and association?

Association is any relationship between two classes (a Teacher and a Student). **Aggregation** is a weak "has-a" where the part can exist alone (a Department has Teachers). **Composition** is a strong "has-a" where the part can't exist without the whole (a House has Rooms). Composition is often preferred over inheritance for flexibility.

Exceptions

Q22. How do you handle exceptions in Java? ★ Most asked

```
try {
  int r = 10 / 0;
} catch (ArithmeticException e) {
  System.out.println("Error: " + e.getMessage());
} finally {
  System.out.println("Always runs");
}
```

`throw` raises an exception; `throws` in a method signature declares that the method may throw one. Use **try-with-resources** for auto-closing: `try (FileReader fr = new FileReader("a.txt")) { ... }`.

Q23. What is the difference between checked and unchecked exceptions? ★ Most asked

- **Checked** exceptions are checked at compile time; you must handle or declare them. Examples: `IOException`, `SQLException`.
- **Unchecked** exceptions (subclasses of `RuntimeException`) happen at runtime and aren't forced to be handled. Examples: `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ArithmeticException`.
- **Errors** (like `OutOfMemoryError`, `StackOverflowError`) are serious problems you usually shouldn't catch.

Q24. How do you create a custom exception?

Extend `Exception` (checked) or `RuntimeException` (unchecked).

```
class InsufficientBalanceException extends Exception {
  InsufficientBalanceException(String msg) { super(msg); }
}
```

Collections

Q25. What is the Collections Framework? ★ Most asked

A set of interfaces and classes for storing and working with groups of objects.

- `List` (ordered, allows duplicates): `ArrayList`, `LinkedList`.
- `Set` (unique items): `HashSet`, `LinkedHashSet`, `TreeSet`.
- `Queue`/`Deque`: `PriorityQueue`, `ArrayDeque`.
- `Map` (key-value, not part of the Collection interface): `HashMap`, `LinkedHashMap`, `TreeMap`.

Q26. What is the difference between ArrayList and LinkedList? ★ Most asked

`ArrayList` uses a dynamic array: fast random access O(1), slow inserts/deletes in the middle O(n). `LinkedList` uses a doubly linked list: slow access O(n), fast insert/delete at the ends or at a known position. ArrayList is the usual default.

Q27. How does HashMap work internally? ★ Most asked

HashMap stores entries in an array of **buckets**.

1. On `put(key, value)`, it calls `key.hashCode()`, mixes the hash and finds the bucket index.

2. If the bucket is empty, it stores the entry there.

3. If another key is already there (a **collision**), it uses `equals()` to check whether it's the same key (then replaces the value) or adds the new entry to the bucket's linked list.

4. Since Java 8, if a bucket gets more than 8 entries (and the table has at least 64 buckets), the list becomes a **red-black tree** for O(log n) lookups.

5. When size exceeds capacity × **load factor** (default 16 × 0.75), the table doubles and entries are rehashed.

Average get/put is O(1). HashMap allows one null key and is not thread-safe.

Q28. What is the difference between HashMap, LinkedHashMap and TreeMap? ★ Most asked

`HashMap` has no order and is fastest. `LinkedHashMap` keeps insertion order. `TreeMap` keeps keys **sorted** (red-black tree, O(log n)).

Q29. What is the difference between HashMap and Hashtable / ConcurrentHashMap?

`Hashtable` is old and fully synchronized (slow) and allows no null keys or values. `HashMap` isn't thread-safe. `ConcurrentHashMap` is the modern thread-safe choice that locks only small parts, so it's much faster than Hashtable.

Q30. What is the difference between Comparable and Comparator?

`Comparable` defines the natural order inside the class itself with `compareTo()` (e.g. sort students by roll number). `Comparator` defines a separate, custom order with `compare()`, so you can have many sort orders.

```
list.sort(Comparator.comparing(Student::getName));
```

Q31. What is the difference between fail-fast and fail-safe iterators?

Fail-fast iterators (ArrayList, HashMap) throw `ConcurrentModificationException` if the collection is changed while looping (except through the iterator's own `remove()`). Fail-safe iterators (ConcurrentHashMap, CopyOnWriteArrayList) work on a copy or a safe view and don't throw.

Multithreading

Q32. How do you create a thread in Java? ★ Most asked

1. Extend the `Thread` class and override `run()`.

2. Implement `Runnable` and pass it to a `Thread` (preferred, since your class can still extend another class).

```
Runnable task = () -> System.out.println("Running in " +
Thread.currentThread().getName());
Thread t = new Thread(task);
t.start();   // start() creates a new thread; calling run() directly would not
```

In real apps, use an `ExecutorService` thread pool.

Q33. What is synchronization? What is a deadlock? ★ Most asked

Synchronization (`synchronized` method or block) lets only one thread access shared code or data at a time, preventing race conditions. A **deadlock** is when two or more threads wait forever for locks held by each other. Avoid it by always acquiring locks in the same order, using timeouts (`tryLock`), and keeping synchronized blocks small.

Q34. What is the difference between a process and a thread?

A process is an independent running program with its own memory. A thread is a lightweight unit of execution inside a process; threads of the same process share memory, which makes communication fast but requires synchronization.

Q35. What does the `volatile` keyword do?

It makes sure every thread reads a variable's latest value from main memory instead of a cached copy. It gives visibility but not atomicity; for `count++` across threads, use `AtomicInteger` or synchronization.

Q36. What is the difference between `wait()` and `sleep()`?

`wait()` (from Object) releases the lock and waits until another thread calls `notify()`; it must be called inside a synchronized block. `sleep()` (from Thread) pauses the thread for a time but does **not** release any lock.

Memory and Java 8+

Q37. How is memory managed in Java? What is garbage collection? ★ Most asked

The JVM divides memory into the **stack** (method calls, local variables, references; one per thread) and the **heap** (all objects, shared). The **garbage collector** automatically finds objects that are no longer reachable and frees their memory, so there are no manual `delete` calls. The heap is split into generations (young and old), since most objects die young. `System.gc()` only *requests* GC.

Q38. What are the main features of Java 8? ★ Most asked

Lambda expressions, functional interfaces, the **Stream API**, default and static methods in interfaces, the `Optional` class, method references (`String::length`) and the new Date/Time API (`java.time`).

Q39. What is a lambda expression and a functional interface? ★ Most asked

A **functional interface** has exactly one abstract method (e.g. `Runnable`, `Comparator`, `Predicate`, `Function`), and can be marked `@FunctionalInterface`. A **lambda** is a short way to implement it: `(a, b) -> a + b`.

Q40. What is the Stream API? ★ Most asked

Streams process collections in a functional style with operations like `filter`, `map`, `sorted`, `collect`, `reduce`. They don't change the original collection and can run in parallel.

```
List<String> names = students.stream()
    .filter(s -> s.getMarks() > 80)
    .map(Student::getName)
    .sorted()
    .collect(Collectors.toList());
```

Intermediate operations (filter, map) are lazy and run only when a terminal operation (collect, forEach, count) is called.

Q41. What is `Optional`?

A container that may or may not hold a value, used to avoid `NullPointerException`. `Optional.ofNullable(user).map(User::getName).orElse("Guest")`.

Q42. What are records (Java 16+)?

A short way to create immutable data classes: `record Point(int x, int y) {}` automatically creates the constructor, getters, `equals`, `hashCode` and `toString`.

Java for web and design

Q43. What is Spring Boot? ★ Most asked

Spring Boot is a framework built on Spring that makes it quick to create production-ready Java applications and REST APIs. It gives auto-configuration, an embedded server (Tomcat), starter dependencies, and minimal setup. A simple REST controller:

```
@RestController
@RequestMapping("/api/users")
public class UserController {
  @GetMapping("/{id}")
  public User get(@PathVariable Long id) { return service.find(id); }
}
```

Q44. What is dependency injection?

Instead of a class creating the objects it needs, they're **provided (injected)** from outside, usually by a framework like Spring through constructors. This makes code loosely coupled and easy to test (you can inject mock objects).

Q45. What are Servlets?

A Servlet is a Java class that runs on a web server and handles HTTP requests and responses (`doGet`, `doPost`). It's the low-level base that frameworks like Spring MVC are built on.

Q46. What are the SOLID principles?

- **S**ingle Responsibility: one class, one job.
- **O**pen/Closed: open for extension, closed for modification.
- **L**iskov Substitution: a subclass should work anywhere its parent is used.
- **I**nterface Segregation: many small interfaces are better than one big one.
- **D**ependency Inversion: depend on abstractions (interfaces), not concrete classes.

Q47. What is the Singleton pattern?

A design pattern that allows only one instance of a class. Make the constructor private, keep a static instance and give a static `getInstance()` method. Used for config managers and loggers.

Q48. What is JDBC?

Java Database Connectivity is the API to connect Java programs to databases: get a `Connection`, create a `PreparedStatement`, execute the query and read the `ResultSet`. `PreparedStatement` also prevents SQL injection. In Spring apps, JPA/Hibernate is usually used on top.

Q49. What is the difference between `throw` and `throws`?

`throw` is used inside a method to actually raise an exception: `throw new IllegalArgumentException("bad")`. `throws` is used in the method signature to declare which checked exceptions the method might pass to the caller: `void read() throws IOException`.

More frequently asked

Q50. What is the difference between `ArrayList` and `Vector`?

Both are dynamic arrays. `Vector` is synchronized (thread-safe but slower) and grows by 100%, while `ArrayList` isn't synchronized (faster) and grows by 50%. `Vector` is a legacy class; use `ArrayList` (or `CopyOnWriteArrayList` for thread safety).

Q51. What is the difference between `HashSet` and `TreeSet`?

`HashSet` stores unique elements with no order and O(1) average operations. `TreeSet` stores them in sorted order (red-black tree) with O(log n) operations and doesn't allow null.

Q52. Can we override a static method in Java?

No. Static methods belong to the class, so a child class with the same static method only **hides** the parent's method (method hiding). Which one is called depends on the reference type, not the object.